Literal Canonicalization in SKJS: Discharging Assumption **L-LIT** (Deep Dive G)

Models revision `bafdcf7c` of `main` (2026-08-05).
Companion to *Refinement and Inference Growth in SKJS* (the main paper).

Abstract

The main soundness paper treats literal types abstractly through a denotation δ_k from stored literal spellings to runtime payloads, and records three claims about it as assumption **L-LIT**: (i) δ is a function, (ii) distinct (normalized) spellings denote $\approx$-inequivalent values, and (iii) the implementation's lexeme-level falsiness test agrees with semantic truthiness. We define the denotation and canonical form of each literal kind, then prove all three claims for canonical forms. Numeric and bigint literals are parsed and rendered canonically by `normalizeJSNumber`/`normalizeJSBigInt`; string escapes and line continuations are decoded to their runtime string value by `JSStringEscape.sk`.

The string results hold, as do the numeric and bigint results within the range where conversion to a runtime number and back preserves the spelling. Beyond that range, distinct numeric spellings can denote the same double, and a nonzero-looking spelling can underflow to zero. These cases break equality narrowing and truthiness reasoning (**L-LIT**). A separate enum-folding failure concatenates a raw numeric operand spelling rather than its runtime string value, producing an incorrect member literal type (**L-LIT-FOLD**). The paper also checks every literal-production path, including template literal type reduction, enum members, numeric property names, set operations, and pretty-printing. Finally, it records two intentional differences from tsc: ECMAScript numeric-string magnitude bounds and rejection of lone-surrogate string escapes under the model's scalar-value domain.

Contents

1 Purpose **2**

2 Recap of Definitions Used **3**

3 The Implementation Today, and the Gap **4**
3.1 Storage and comparison 4
3.2 Construction sites of literal lexemes 4
3.3 The falsiness test 5
3.4 The witnessed violation 5

4 Denotations δ_k **5**

5 Canonical Forms **6**
5.1 Numbers 6
5.2 Bigints 7
5.3 Strings, booleans, symbols 7

6 Proofs over Canonical Forms **8**

7 Canonicalization **9**
7.1 Where normalization lives . . . 9
7.2 Strings: decoding at the quote boundary . . . 10
7.3 Template literal types concatenate canonical lexemes . . . 10
7.4 Enum member values enter the literal domain . . . 11
7.5 Validation of over-lexed input . . . 12
7.6 User-visible pretty-printing . . . 12
7.7 Interaction with `PrimLiterals` operations . . . 13
7.8 Numeric property names . . . 13
7.9 Falsiness on canonical lexemes . . . 13

8 Findings from Failed and Partial Proof Attempts **13**

9 Status and Registry **15**
9.1 L-LIT at the pin . . . 15
9.2 Registry . . . 15
9.3 Registry alignment . . . 16
9.4 Pinned tests and executable witnesses . . . 16

1 Purpose

This paper serves the assumptions-and-obligations ledger of the main paper (its Section "Assumptions and Proof Obligations") as follows.

- **L-LIT(i)** (δ is a function): PROVED (definitional). This paper *strengthens* the row: we give δ_k concretely for every kind (Section 4), prove that no numeric literal denotes `NaN` (which the main paper's Definition of Literal Denotation assumes), and verify that the string decoder realizes $\delta_{\texttt{str}}$ on everything it accepts (Section 7.2).

- **L-LIT(ii)** (injectivity modulo $\approx$): this paper defines the canonical forms and *proves* injectivity-modulo-$\approx$ over them (Theorem 14). For `str` it HOLDS outright — stored lexemes are decoded string values, so the denotation restricted to storage is the identity (Section 7.2). For num/bigint it HOLDS on the within-precision range — the parser normalizes literals (Section 7) — and is VIOLATED for beyond-precision numeric spellings, which alias one double yet stay distinct (Finding 22).

- **L-LIT(iii)** (falsiness-exactness): this paper proves falsiness-exactness over canonical forms (Theorem 15), giving the exact finite falsy canonical set per kind. For `str` it HOLDS: the only falsy string is `""`, and a lexeme is stored empty exactly when its source spelling denotes the empty string (line continuations are elided by the decoder). For num/bigint it HOLDS within precision and is VIOLATED beyond it (a normalized underflow lexeme denotes +0 yet is classified truthy, Finding 22).

Downstream, the string-kind and within-precision num/bigint discharge of (ii) and (iii) restores: the literal-filtering case of the main paper's Lemma "Residues are Sound" (the one soundness-relevant caveat in that document), and the exactness clause of its Theorem "Soundness of Constant Removal" — each on the canonical range; for beyond-precision `num` spellings the caveat persists.

No theorem statement or proof in the main paper depends on this; the violation is local to the leaf assumption, exactly as its summary asserts.

Numeric and bigint literals are normalized in the parser (`getNumber`/`getBigInt`, with `negateJSNumberLiteral` folding -0); string literals are decoded in the parser (`unquoteString`/`cookTemplatePart` over `decodeJSStringLiteral`). Section 7 describes both and Section 9 the ledger row. The beyond-precision `num` gap is Finding 22.

2 Recap of Definitions Used

We reuse the main paper's runtime model verbatim and re-state only what the proofs below manipulate; everything else is cited by name.

From the main paper, Section "Runtime Model":

- Values are tagged payloads, $\mathsf{Val} = \{(k, p) \mid k \in \mathsf{Tags},\ p \in \mathsf{Payload}_k\}$, with $\mathsf{Payload}_{\mathsf{num}} = \mathbb{F}$ (IEEE-754 doubles, including -0 and NaN), $\mathsf{Payload}_{\mathsf{str}} = \mathsf{String}$, $\mathsf{Payload}_{\mathsf{bigint}} = \mathbb{Z}$, $\mathsf{Payload}_{\mathsf{bool}} = \{\mathsf{true}, \mathsf{false}\}$, $\mathsf{Payload}_{\mathsf{sym}} = \mathsf{SymVal}$.
- Strict equality $\approx$: same tag and equal payloads, except that on numbers -0 is identified with 0 and NaN is equal to nothing, and the $\mathsf{undef}/\mathsf{void}$ tags are related cross-tag (irrelevant here: no literal type has a unit tag).
- Truthiness (main paper, Definition "Truthiness"): (num, n) is falsy iff $n \in \{0, -0, \mathsf{NaN}\}$; (str, s) falsy iff $s =$ `""`; (bigint, z) falsy iff $z = 0$; (bool, b) falsy iff $b = \mathsf{false}$; symbols always truthy.
- Literal types: $\mathsf{prim}(k, L)$ with $L \subseteq \mathsf{Lit}_{\mathtt{k}}$ finite nonempty, and the *same-tag* $\approx$*-closure* semantics
$$[\![\mathsf{prim}(k, L)]\!] = \{\, v \mid \mathsf{rtag}(v) = \mathsf{tag}(k),\ \exists \ell \in L.\ v \approx (\mathsf{tag}(k), \delta_k(\ell)) \,\}$$
so the literal type `0` contains both 0 and -0, and $\mathsf{Lit}_{\mathtt{num}}$ is assumed to contain no NaN spelling.

For numbers we write $\doteq$ for the restriction of $\approx$ to num payloads: $x \doteq y$ iff x and y are equal as IEEE doubles with -0 identified with 0; $\mathsf{NaN} \not\doteq \mathsf{NaN}$. On $\mathbb{F} \setminus \{\mathsf{NaN}\}$, $\doteq$ is an equivalence whose only non-singleton class is $\{0, -0\}$.

Assumption L-LIT, as stated in the main paper's ledger. (i) δ is a function — definitional, used by the unconditional part (R2) of constant-removal soundness; (ii) *injectivity modulo* $\approx$ after normalization — distinct normalized lexemes denote $\approx$-inequivalent values; needed only for *exactness* claims; (iii) *falsiness-exactness* — the lexeme test `isFalsy` agrees with the Definition of Truthiness on every stored literal; needed by the residue lemma. Anchors: `normalizeJSNumber`/`normalizeJSBigInt`; `isFalsy`; the string-escape decoding (`JSStringEscape.sk`). The row's per-kind status is maintained in the registry and rendered in Section 9.

3 The Implementation Today, and the Gap

3.1 Storage and comparison

A literal is stored as a bare string with a position; identity, ordering, hashing, and all set operations are by that string: `PrimLiteral(pos, val)` projects to `val` and compares by it; `PrimLiterals` is a string-keyed array map whose `union/inter/diff/contains` are exact set operations *on lexemes*, and whose `xxhash` hashes only the strings (positions excluded).

impl: `PrimLiterals.sk / PrimLiteral, PrimLiterals (union, inter, diff, contains, xxhash)`

The kind (`num`, `bigint`, `str`, `bool`, `sym`) lives one level up, on `TPrimValue(kind, literals)`; `PrimLiteral` itself is kind-blind. Lexeme-level operations are therefore exact value-set operations exactly where stored lexemes are canonical — one spelling per denoted value — which is what parse-time canonicalization establishes per kind (Section 7), except on the beyond-precision `num` residue (Finding 22).

impl: `TType.sk / TPrimValue, TPrimValueKind`

3.2 Construction sites of literal lexemes

Every path that creates a numeric or bigint literal lexeme:

Site	Lexeme created
`ParserUtils.sk / getNumber, getBigInt` (lines 22–44)	the normalized canonical form (`validate*` then `normalizeJSNumber/normalizeJSBigInt`)
`ParserType.sk / parsePrimary`	type annotations `TANumLiteral/TABigIntLiteral`; negative forms fold through `negateJSNumberLiteral` (lines 232–234)
`TEnv.sk / normalize`	converts those AST literals to `TPrimValue::single` unchanged
`TypeCheckExpr.sk / directLiteralSType`	expression literals (`Number`, `BigInt` tokens); unary `-` folds through `negateJSNumberLiteral` (:3534)
`TypeCheckExpr.sk / asConstExpr` (`UOMinus` arm, :4707)	sign-flips through `negateJSNumberLiteral`
`TEnv.sk` line 3315 (`keyof`)	string keys passing `isJSNumericString` become number literals with the same string
`TypeCheckEnum.sk` (fold results; §7.4)	exact integer folds rendered by `Int.toString()` (canonical decimal); non-exact folds create *no* literal; the string-concat arms create canonical strings that need not match the runtime value (Finding 21, row L-LIT-FOLD)
`SyntheticTuples.sk` (lines 116, 127)	tuple arity via `Int.toString()` (canonical decimal)
`TType.sk / truthinessValueResidue`	the constants `"0n"`, `""`

Every numeric and bigint site routes through the shared normalizer (`getNumber/getBigInt`) or creates a canonical decimal directly, with the negative forms folded by `negateJSNumberLiteral`. Consequently the stored lexeme set for a source union `0x1 | 1` has *one* element, `"1"`, matching the semantic singleton.

String literal lexemes have their own construction sites, all routed through the escape decoder; Section 7.2 inventories them.

3.3 The falsiness test

Truthiness classification of a literal is by lexeme: `TPrimValueKind::isFalsy` answers `literal == "false"` (bool), `literal == ""` (str), always-false (sym), and for num/bigint compares against the canonical zero: `literal == "NaN" || literal == "0"` (num) and `literal == "0n"` (bigint). The string arm is exact: a stored string lexeme is the decoded runtime value, so it is empty exactly when the value is (in particular a line-continuation spelling is stored as `""`, since the decoder elides it). The num/bigint arms are *exact on the within-precision range* because numeric and bigint literals there are normalized to canonical form at parse time (`getNumber`/`getBigInt`, Section 7), so zero has exactly one *grammatical* spelling per kind: the zero spellings `0.0`, `0e0`, `0x00` and their bigint forms fold to `"0"`/`"0n"` (the forms `0_0` and `00` are rejected outright — `NumericSeparatorsNotAllowedHere`, `OctalLiteralsNotAllowed`). Beyond precision the equality test is *not* exact: a normalized underflow lexeme such as `1e-400` denotes +0 yet fails the `== "0"` test (Finding 22). The num arm retains the `"NaN"` check — defensive only, since no construction site of Section 3.2 can create it (Theorem 4 makes this precise).

impl: `TType.sk / TPrimValueKind::isFalsy (:1548--1556)`

Three consumers read this classification: `SType::falsiness` counts falsy literals per member (feeding the `safe`-mode condition guard in `TypeCheckUtils.sk / checkCondition` (:144), which requires "boolean or a type with a single falsy value"); `SType::truthinessResidue` filters literals by `isFalsy` for `if`/`else` narrowing; and `truthinessValueResidue` does the same for the value of `&&`/`||`.

impl: `TType.sk / SType::falsiness (:1729), truthinessResidue (:1774), truthinessValueResidue (:1797); TypeCheckUtils.sk / checkCondition`

3.4 The witnessed violation

The live **L-LIT** violation is num-kind, on the beyond-precision residue: spellings aliasing one double are stored distinct, breaking (ii) — equality narrowing types a live branch `never` — and a normalized underflow lexeme (`1e-400`) denotes +0 yet is classified truthy, breaking (iii). Both are witnessed end-to-end in Finding 22.

Elsewhere no such witness exists. Parse-time canonicalization gives one spelling per value, so `falsiness` never double-counts a repeated zero and the stored lexeme set of a union is injective over its classes (Theorems 15, 14). A union like `0.0 | 0x0` stores the single lexeme `"0"`; a union `"a" | "\x61"` stores the single lexeme `"a"`; a condition `0 | ""` is rejected as a two-falsy-value condition — including when the `""` member is spelled as a line continuation — and `0 | "a"` narrows correctly, all exactly per the model. Bigint is exact at every magnitude (`radixToDecimalString` has no precision limit). The pinned test `acceptOctalZero` (`tests/projects/truthiness-safe/src/conditions.ts`, line 109) exercises `0o0 | 1`, which normalizes to `0 | 1` and narrows soundly; the string decoder's behavior is pinned by a dedicated suite (Section 9.4).

4 Denotations δ_k

The main paper assumes a denotation $\delta_k : \mathsf{Lit_k} \to \mathsf{Payload}_{\mathsf{tag}(k)}$. We now fix it per kind. $\mathsf{Lit_k}$ is the set of strings that can be stored for kind k by the construction sites of Section 3.2; for num/bigint these are ECMAScript *NumericLiteral*/*BigIntLiteral* lexemes optionally prefixed by `-` (the parser composes the sign), for str the *decoded* string value (Section 7.2), for bool {`"true"`, `"false"`}, and for sym synthesized identities.

Definition 1 ($\delta_{\texttt{num}}$)**.** *For a NumericLiteral body ℓ (after removing any leading* `-` *and all digit separators* `_`*): its* mathematical value $\mathrm{MV}(\ell) \in \mathbb{R}_{\geq 0}$ *is given by the ECMA-262 static semantics of NumericLiteral (decimal, or radix* 16/8/2 *under* `0x/0o/0b` *prefixes, exponents included), and $\delta_{num}(\ell)$ is* the Number value for $\mathrm{MV}(\ell)$*: the IEEE-754 round-to-nearest-ties-to-even image of that real in $\mathbb{F}$, with overflow to $+\infty$. A leading* `-` *negates the result (giving -0 when the body denotes 0). We also admit the two words* `Infinity/-Infinity` *with denotations $\pm\infty$ (they arise only as canonical renderings, never as source lexemes —* `Infinity` *lexes as an identifier).*

Definition 2 ($\delta_{\texttt{bigint}}$)**.** *For a BigIntLiteral $\ell = mn$ (optionally* `-`*-prefixed), $\delta_{bigint}(\ell) \in \mathbb{Z}$ is the signed integer value of m in its radix, separators removed. Exact; no rounding.*

Definition 3 ($\delta_{\texttt{str}}, \delta_{\texttt{bool}}, \delta_{\texttt{sym}}$)**.** *$\delta_{str}(\ell)$ is the string value ℓ denotes. Because the parser stores the* decoded *value (escape sequences decoded, line continuations elided, per ECMA-262 §12.9.4.2; Section 7.2), the stored lexeme* is *that value and δ_{str} restricted to storage is the identity. δ_{bool} maps* `"true"/"false"` *to* true/false*. δ_{sym} maps each synthesized position-derived identity (*`unique_symbol:file:sym:offset:name`*) to a distinct element of* SymVal*; as the main paper notes, these are not source lexemes, so distinct stored strings have distinct denotations* by construction *— kind* `sym` *is already canonical and plays no further role here.*
impl: `TEnv.sk / uniqueSymbol`

NaN is not in the range of $\delta_{\texttt{num}}$:

Theorem 4 (No NaN literal)**.** *For every $\ell \in \mathsf{Lit}_{num}$, $\delta_{num}(\ell) \neq \mathsf{NaN}$.*

Proof. $\mathrm{MV}(\ell)$ is a nonnegative real; the Number value of a real is a finite double or $\pm\infty$, never NaN, and negation preserves that. The only non-grammar lexemes admitted are `Infinity`/`-Infinity`, denoting $\pm\infty$. The remaining creation paths — `keyof` over string keys, which filters through `isJSNumericString` (a grammar with no `NaN` production), and the integer renderings of the enum folder and `SyntheticTuples` — create only numerals. □

This *proves* (rather than assumes) the main paper's stipulation that $\mathsf{Lit}_{\texttt{num}}$ contains no NaN spelling, and shows the `literal == "NaN"` arm of `isFalsy` is defensive: no construction site creates `NaN`.
impl: `JSNumericString.sk / isJSNumericString; TType.sk / isFalsy`

5 Canonical Forms

The canonicalization strategy is uniform: *parse the lexeme to the denoted value, then re-render that value by a fixed canonical rendering.* Injectivity then reduces to properties of the rendering, independent of how baroque the accepted input grammar is.

5.1 Numbers

The canonical rendering is JavaScript's own `Number::toString` (radix 10), written $\mathsf{render}_{\texttt{num}} : \mathbb{F} \setminus \{\mathsf{NaN}\} \to \mathsf{String}$. We treat IEEE-754 and the ECMA-262 rendering algorithm as an abstract interface with three axioms (the "abstracted-with-axioms" bin of the modeling contract); each is a stated property of ECMA-262 §6.1.6.1.20 (*Number::toString*).

Axiom 5 (NUM-FUN)**.** *render_{num} is a total function on $\mathbb{F} \setminus \{\mathsf{NaN}\}$, with $\mathsf{render}_{num}(\pm\infty) =$* `"Infinity"/"-Infinity"`*.*

Axiom 6 (NUM-RT (round-trip))**.** *For every finite x, $\delta_{num}(\mathsf{render}_{num}(x)) \doteq x$. (ECMA-262 chooses digits s and exponent n such that* the Number value for $s \cdot 10^{n-k}$ *is x; parsing the rendered string recovers exactly that value. For $x \doteq 0$ the rendering is* `"0"`*, which parses to $+0 \doteq x$.) For $x = \pm\infty$ the round-trip is by the* `Infinity` *clause of Definition 1.*

Axiom 7 (NUM-ZID (zero identification))**.** $x \doteq y \implies \mathsf{render}_{num}(x) = \mathsf{render}_{num}(y)$. *(The only non-singleton $\doteq$-class is $\{0, -0\}$, and the algorithm returns* `"0"` *for both — the sign of -0 is dropped.)*

Definition 8 (Canonical numeric lexemes; $\mathsf{canon}_{\mathtt{num}}$)**.** $\mathsf{Canon}_{num} \triangleq \mathsf{render}_{num}(\mathbb{F} \setminus \{\mathsf{NaN}\})$*, and* $\mathsf{canon}_{num} \triangleq \mathsf{render}_{num} \circ \delta_{num} : \mathsf{Lit}_{num} \to \mathsf{Canon}_{num}$.

Justification of the rendering choice. (1) $-0 \mapsto$ `"0"` (Axiom NUM-ZID) implements the main paper's $\approx$-closure semantics of literal types — the literal type `0` already denotes $\{0, -0\}$, so a distinct `-0` canonical spelling would be a second name for the same type; tsc likewise gives `const x = -0` the type `0`. (2) $\mathsf{NaN} \notin \mathrm{dom}(\mathsf{render}_{\mathtt{num}})$ matches the main paper's exclusion of a NaN literal type (its denotation would be $\emptyset$ under closure semantics), and Theorem 4 shows totality is not thereby lost. (3) The finite part of $\mathsf{Canon}_{\mathtt{num}}$ is exactly the strings satisfying $\mathtt{String}(\mathtt{Number}(\mathtt{s})) = s$, the predicate the codebase already uses to classify numeric property names — so lexemes created by `keyof` number literals and canonicalized literals agree by construction (Finding 23 details how the predicate enforces the magnitude bounds). (4) It is what tsc renders in hovers and errors, minimizing user-visible divergence.

impl: `JSNumericString.sk / isJSNumericString (intended grammar)`

Note $\mathsf{Canon}_{\mathtt{num}}$ contains renderings such as `"1e-7"` whose signed exponent the SKJS lexer cannot tokenize as a single numeric token (Section 7.5); this is harmless because stored lexemes are never re-lexed, and tsc accepts these spellings.

5.2 Bigints

Here everything is concrete; no axioms are needed.

Definition 9 (Canonical bigint lexemes)**.** $\mathsf{render}_{\mathbb{Z}}(z)$ *is the standard signed decimal numeral of $z \in \mathbb{Z}$ (no leading zeros, no separators, no radix prefix, no sign for $z \geq 0$, so* $\mathsf{render}_{\mathbb{Z}}(0) =$ `"0"` *— in particular* `-0n` *normalizes to* `0n`*), and* $\mathsf{Canon}_{bigint} \triangleq \{\mathsf{render}_{\mathbb{Z}}(z)\,\mathtt{n} \mid z \in \mathbb{Z}\}$*, with* $\mathsf{canon}_{bigint} = (\mathsf{render}_{\mathbb{Z}} \circ \delta_{bigint})\,\mathtt{n}$.

Fact 10. $\mathsf{render}_{\mathbb{Z}}$ *is a bijection $\mathbb{Z} \to \mathsf{render}_{\mathbb{Z}}(\mathbb{Z})$, and $\delta_{bigint}(\mathsf{render}_{\mathbb{Z}}(z)\,\mathtt{n}) = z$. (Uniqueness of positional decimal numerals without leading zeros; elementary.)*

5.3 Strings, booleans, symbols

$\mathsf{Canon}_{\mathtt{str}} \triangleq \mathsf{String}$: the canonical form of a string literal is *the denoted string itself* ($\mathsf{canon}_{\mathtt{str}} = \delta_{\mathtt{str}}$: decode escapes at the quote boundary and store the value). The parser realizes exactly this (Section 7.2), on the scalar-value subset of String (Finding 24). $\mathsf{Canon}_{\mathtt{bool}} = \{\mathtt{"true"}, \mathtt{"false"}\}$ and $\mathsf{Canon}_{\mathtt{sym}}$ = the synthesized identities: already canonical, canon is the identity on them (Definition 3).

Definition 11 (Canonical-storage invariant)**.** *A* `TPrimValue(kind k, Some(lits))` *is* canonical *when every stored lexeme is in Canon_k. Parse-time canonicalization (Section 7) establishes this outright for* `str`*,* `bool`*,* `sym`*, and bigint, and for num except* beyond-precision *literals — those failing the round-trip* $\mathtt{String}(\mathtt{Number}(\mathtt{s})) = s$*: 16–17-digit decimals, radix forms $> 2^{53}$, over/underflow*

exponent forms — which are stored non-canonically (some in spelled form, some rewritten: `1e309` becomes `"1e+309"`, a longhand 10^{-400} becomes `"1e-400"`). Distinct such lexemes are never merged; that keeps subsumption sound but is itself the defect wherever a lexeme-level $\bot$ or falsiness verdict is read semantically (Finding 22).

6 Proofs over Canonical Forms

Throughout this section literals are canonical (Definition 11); kind `sym` is covered by construction and omitted from the case analyses where it is trivial.

Lemma 12 (Idempotence)**.** *For $k \in \{\texttt{num}, \texttt{bigint}\}$ and $\ell \in \mathsf{Canon}_k$: $\mathsf{canon}_k(\ell) = \ell$.*

Proof. `num`: $\ell = \mathsf{render}(x)$ for some x; then $\mathsf{canon}(\ell) = \mathsf{render}(\delta(\mathsf{render}(x)))$, and by NUM-RT $\delta(\mathsf{render}(x)) \doteq x$, so by NUM-ZID $\mathsf{render}(\delta(\mathsf{render}(x))) = \mathsf{render}(x) = \ell$ (infinite x: direct). `bigint`: immediate from Fact 10. □

Theorem 13 (L-LIT(i): δ is a function; totality)**.** *Each δ_k is a well-defined function on Lit_k, and each canon_k is a well-defined function $\mathsf{Lit}_k \to \mathsf{Canon}_k$.*

Proof. `num`: MV is defined by a deterministic grammar attribution; rounding is a function; Axiom NUM-FUN gives the rendering (Theorem 4 keeps NaN out of its domain). `bigint`: radix valuation and Fact 10. `str`: escape decoding is deterministic (`decodeJSStringLiteral` is a single left-to-right pass with no choice points), and on stored lexemes δ is the identity. `bool`, `sym`: finite/by-construction tables. □

Theorem 14 (L-LIT(ii): injectivity modulo $\approx$)**.** *For every kind k and $\ell_1, \ell_2 \in \mathsf{Canon}_k$: if $(\mathsf{tag}(k), \delta_k(\ell_1)) \approx (\mathsf{tag}(k), \delta_k(\ell_2))$ then $\ell_1 = \ell_2$. Equivalently: distinct canonical lexemes denote $\approx$-inequivalent values, so each $\approx$-class of payloads meets at most one canonical lexeme.*

Proof. `num`: on num payloads $\approx$ is $\doteq$. Write $\ell_i = \mathsf{render}(x_i)$. By NUM-RT $\delta(\ell_i) \doteq x_i$ (for infinite x_i, $\delta(\ell_i) = x_i$), so $\delta(\ell_1) \doteq \delta(\ell_2)$ gives $x_1 \doteq x_2$ by symmetry/transitivity of $\doteq$ on its domain (NaN is not in the range of δ by Theorem 4). NUM-ZID then gives $\ell_1 = \mathsf{render}(x_1) = \mathsf{render}(x_2) = \ell_2$. `bigint`: $\approx$ is equality on $\mathbb{Z}$; Fact 10 (bijectivity) concludes. `str`: $\mathsf{canon}_{\texttt{str}}$ stores the value itself, so δ restricted to $\mathsf{Canon}_{\texttt{str}}$ is the identity — injective. `bool`: two-point table, injective. `sym`: by construction (main paper, Definition "Literal Denotation"). □

Theorem 15 (L-LIT(iii): falsiness-exactness)**.** *Define the* falsy canonical sets

$$F_{\texttt{bool}} = \{\texttt{"false"}\}, \quad F_{\texttt{num}} = \{\texttt{"0"}\}, \quad F_{\texttt{str}} = \{\texttt{""}\}, \quad F_{\texttt{bigint}} = \{\texttt{"0n"}\}, \quad F_{\texttt{sym}} = \emptyset.$$

For every kind k and $\ell \in \mathsf{Canon}_k$: $\mathsf{truthy}(\mathsf{tag}(k), \delta_k(\ell)) = \mathsf{false} \iff \ell \in F_k$. Hence over canonical forms the lexeme test "$\ell \in F_k$" agrees exactly with the main paper's Definition of Truthiness, and each F_k is finite (at most one element).

Proof. By the Definition of Truthiness, the falsy payloads are: false (bool); $\{0, -0, \mathsf{NaN}\}$ (num); `""` (str); 0 (bigint); none (sym). `num`: $\mathsf{NaN} \notin \mathrm{range}(\delta)$ (Theorem 4); $\delta(\ell) \in \{0, -0\}$ iff $\delta(\ell) \doteq 0$ iff (by Lemma 12 and NUM-ZID) $\ell = \mathsf{canon}(\ell) = \mathsf{render}(\delta(\ell)) = \mathsf{render}(0) =$ `"0"`; conversely $\delta($`"0"`$) = 0$ is falsy. `bigint`: $\delta(\ell) = 0$ iff $\ell =$ `"0n"` by Fact 10. `str`: identity denotation, falsy iff the stored string is empty. `bool`, `sym`: by the tables. □

Corollary 16 (Residue lemma discharged)**.** *With canonical storage and* `isFalsy` *realized as membership in F_k (a single string equality per kind), the literal-filtering case of the main paper's Lemma "Residues are Sound" holds unconditionally: the kept-literal sets of tres_b and vres_b contain exactly the literals whose denotations have truthiness b.*

impl: `TType.sk / truthinessResidue, truthinessValueResidue`

Corollary 17 (Exactness of constant removal)**.** *The exactness clause of the main paper's Theorem "Soundness of Constant Removal" holds under canonical storage: its stated side condition is precisely L-LIT(ii), Theorem 14. (Its soundness clauses R1/R2 were unconditional already.)*

impl: `TType.sk / removeConstant; PrimLiterals.sk / diff`

Corollary 18 (Lexeme set operations compute value set operations)**.** *For canonical $L_1, L_2 \subseteq \mathsf{Canon}_k$, writing $[\![L]\!] \triangleq [\![\mathsf{prim}(k, L)]\!]$: $[\![L_1 \cup L_2]\!] = [\![L_1]\!] \cup [\![L_2]\!]$ (unconditional); $[\![L_1 \cap L_2]\!] = [\![L_1]\!] \cap [\![L_2]\!]$; and $[\![L_1 \setminus L_2]\!] = [\![L_1]\!] \setminus [\![L_2]\!]$.*

Proof. Union: $[\![\cdot]\!]$ is a union of $\approx$-closures, which commutes with $\cup$. Intersection and difference: by Theorem 14 the $\approx$-classes named by distinct canonical lexemes are disjoint, so a value $v \approx \delta(\ell_1)$ with $\ell_1 \in L_1$ lies in $[\![L_2]\!]$ iff $\ell_1 \in L_2$; the class-wise identities follow. (Without canonicity only $\subseteq$ / $\supseteq$ over-approximations hold, which is why `PrimLiterals::inter` spuriously produces $\bot$ on the beyond-precision lexemes: $\{\texttt{"9007199254740993"}\} \cap \{\texttt{"9007199254740992"}\} = \emptyset$ on lexemes while both spell the double 2^{53} — an unsound *emptiness* verdict wherever $\bot$ is read as disjointness, witnessed in Finding 22.)

impl: `PrimLiterals.sk / union, inter, diff` □

Corollary 19 (Exact falsy counting)**.** *Over canonical storage,* `SType::falsiness` *counts, per literal member, the exact number of falsy $\approx$-classes (0 or 1, since $|F_k| \leq 1$), so* `FalsyCount` *never double-counts a class across spellings and the* `safe`*-mode condition guard implements its intended semantics on literal unions — including unions whose string members are spelled with escapes or line continuations.*

impl: `TType.sk / SType::falsiness, FalsyCount; TypeCheckUtils.sk / checkCondition`

7 Canonicalization

7.1 Where normalization lives

Normalization lives at one choke point, the parser. `getNumber`/`getBigInt` (`ParserUtils.sk:22-44`) call `validateJSNumericLiteral`/`validateJSBigIntLiteral` then `normalizeJSNumber`/`normalizeJSBigInt`. Two consequences of parser placement: (1) the same helpers feed numeric *property names* (`ParserDefs.sk / getNameOrStringOrNumber`, :72), so property names canonicalize too; (2) the negative sites, which compose `"-"` after the helper returns (`ParserType.sk:232,234`; `directLiteralSType`; `asConstExpr`), route through `negateJSNumberLiteral` (`JSNumericString.sk:479`), which folds $-0 \to 0$ and $-0n \to 0n$ over the normalized lexeme. Every num/bigint `PrimLiteral` therefore carries a canonical lexeme by construction, establishing Definition 11 (modulo beyond-precision literals). Boolean and symbol constructors are already canonical, as are the synthesized constants (`SyntheticTuples.sk` 116, 127; `truthinessValueResidue`'s `"0n"`/`""`).

impl: `ParserUtils.sk / getNumber, getBigInt; JSNumericString.sk / normalizeJSNumber (:492), normalizeJSBigInt, negateJSNumberLiteral`

String literals are canonicalized at the same choke point; Section 7.2 gives the decoder.

7.2 Strings: decoding at the quote boundary

Every string-literal lexeme is stored *decoded*: the parser maps the source body to its ECMA-262 String Value (§12.9.4.2) before any type is built, so the stored lexeme equals the runtime string and $\mathtt{canon}_{\mathtt{str}}$ of Section 5.3 is realized by construction. The decoder is `decodeJSStringLiteral` (`JSStringEscape.sk:105-191`): a single left-to-right pass that maps the single-character escapes (`\n t r b f v 0`) to their code points, decodes `\xHH`, `\uHHHH` (combining a surrogate pair into one code point), and `\u{...}`, elides *LineContinuation*s (backslash before LF, CR, CRLF, LS, PS — so a line-continuation literal stores `""`), and passes *NonEscapeCharacter*s through as themselves. Call sites:

Site	What it decodes
`ParserUtils.sk / unquoteString` (:57–73)	quoted string literals: expression literals (`ParserExpr.sk:189`), type annotations (`ParserType.sk:64`), and quoted property names (`ParserDefs.sk:50,67,78`)
`ParserUtils.sk / cookTemplatePart` (:49–55)	no-substitution template expressions (`ParserExpr.sk:628`) and template-literal-type chunks (`ParserType.sk:352,361`), with line terminators first normalized to the TV spelling (`normalizeTemplateLineTerminators`)
`ParserUtils.sk / getVar` (:12–20)	identifiers, via `decodeIdentifierName`, so `caf\u{e9}` and `café` are one name

Interpolated template *expressions* are the deliberate non-consumer: their chunks are validated but kept raw (`ParserExpr.sk / validateTemplatePart`, :659–666), because the expression is typed `string` without its parts becoming values (`TypeCheckExpr.sk:1878`); no literal type is created, so no canonicality obligation arises.

Invalid forms are *rejected*, not guessed, mirroring `validateJSNumericLiteral`: legacy octal escapes and `\8`/`\9` (strict-mode syntax errors), malformed `\x`/`\u`, `\u{}` values above `0x10FFFF`, and lone surrogate escapes (Finding 24). Rejection is the conservative direction: no lexeme is stored whose denotation the decoder cannot represent. One admission gap sits upstream in the lexer: a *raw* lone CR inside a quoted string is accepted as content where ECMAScript calls the string unterminated (upstream #901) — the program is invalid JS and cannot run, so no runtime value contradicts a type, but the acceptance is a parser-parity gap adjacent to this paper's scope.

The decoded lexemes make injectivity and falsiness structural: the union `"a" | "\x61"` stores one lexeme, `===` narrowing against an escape-spelled constant agrees with runtime equality, `inter` on overlapping string literal types is exact (Corollary 18), and the falsiness test's `literal == ""` arm is exact (Theorem 15).

impl: `JSStringEscape.sk / decodeJSStringLiteral, decodeUnicodeEscape, normalizeTemplateLineTerminators, decodeIdentifierName; ParserUtils.sk / unquoteString, cookTemplatePart`

7.3 Template literal types concatenate canonical lexemes

A template literal *type* whose holes all have finitely many values reduces to a finite string-literal union: each hole contributes, per member, the exact runtime `String()` spelling, and the reduction concatenates cooked chunk text with those spellings over the full product (`TEnv.sk / normalizeTemplate`, :2168–2274; `templateHole`, :2277). The per-kind spellings are exact *because* storage is canonical:

- a string-literal member is already the runtime value (§7.2);
- a boolean member contributes `"true"`/`"false"`, and bare `boolean` contributes both;
- a bigint member strips the `n` suffix from its canonical decimal — exact at any magnitude;
- `null`/`undefined` contribute their words;
- a number-literal member contributes its lexeme *only when* `isPreciseJSNumberLiteral` holds — exactly the round-trip predicate `String(Number(s)) = s`, so the lexeme *is* the runtime spelling. A beyond-precision member makes the hole irreducible and the template is rejected (`TEnv.sk:2306--2318`).

Fact 20 (Template reduction is exact)**.** *When the reduction succeeds, the created literal set is* $\{ c_0 v_1 c_1 \cdots v_n c_n \mid v_i \in V_i \}$ *where* c_j *are the cooked chunk values and* V_i *the per-member runtime spellings above — each created lexeme is the exact runtime value of the corresponding member choice, so the created* ***TPrimValue*** *is canonical (Definition 11) and its semantics is exactly the set of strings the template type denotes. Absorption cases are handled before materialization: a* ***never*** *hole gives* $\bot$*; an all-bare-****string*** *template with no text of its own* is ***string****; a product about to exceed* ***kMaxTemplateUnionSize*** *(*10^5*,* ***TEnv.sk:16****) is rejected (TS2590) rather than truncated.*

Two design notes. First, the beyond-precision refusal in the number arm addresses the same precision defect as Finding 22, handled *conservatively* here: where the `===`-narrowing path reads a lexeme-level verdict semantically with no precision guard, the template path refuses to create a lexeme that does not pin down one double. Second, holes that are `number`, `bigint`, or abstract are rejected outright (SKJS has no pattern representation for an infinite-but-partial set of strings), and a hole that may not be interpolated at all (e.g. `symbol`) is a TS2322 error — both rejections, so no unsound literal is created on any template path.

impl: `TEnv.sk / normalizeTemplate, templateHole; JSNumericString.sk / isPreciseJSNumberLiteral (:87)`

7.4 Enum member values enter the literal domain

Enum members create literal types on three routes. Canonicality — is the created lexeme a canonical form? — holds on all three, so every enum member literal is inside **L-LIT**'s scope like any other stored literal, and every registry consumer of L-LIT's form properties covers enum literals unchanged. *Value faithfulness* — does the created literal equal the runtime result of its source expression? — is the *orthogonal* claim, distinctive to synthesis: it holds on the first two routes and fails on the third. It is registry row **L-LIT-FOLD**.

A *direct* initializer (or the implicit auto-increment seed `"0"`) carries a lexeme from the ordinary literal path — normalized numerics, decoded strings — so it inherits every result of this paper, including the beyond-precision exception (a member initialized with a beyond-precision spelling stores that spelling, like any other literal); faithfulness here is definitional — the ordinary literal is carried through unchanged, so the member denotes what the written literal denotes (**L-LIT**(i)), with no reliance on (ii)/(iii), which remain violated beyond precision. A *numerically folded* initializer is computed by exact integer arithmetic gated to the double-safe range: operands must parse by `toIntOption` with magnitude $\leq 2^{53}-1$, every operation checks for overflow and inexactness (division folds only when exact; fractional, overflowing, or out-of-range results widen to an *untracked number*, creating no literal), and results render through `Int.toString()` — canonical decimal numerals denoting exactly the runtime results, so this route is faithful.

Finding 21 (The string-concat fold arms can disagree with runtime values for beyond-precision operands: L-LIT-FOLD is violated)**.** *The third route is string concatenation: the* ***BOAddition*** *arms build* ***EFStr(a + b)*** *for* ***(EFStr, EFStr)****,* ***(EFStr, EFNum)****, and* ***(EFNum, EFStr)*** *by textual concatenation of operand* lexemes*. The numeric precision check in* ***enumNumOrUnknown*** *guards numeric fold* results *only; a direct numeric-literal operand contributes its stored spelling with no precision check. At runtime the engine converts the* value *(****ToString(Number)****), so any operand whose spelling fails the round-trip predicate concatenates wrongly:* ***enum E { A = "x" + 1e-400 }*** *creates the member literal* ***"x1e-400"*** *while* ***E.A*** *evaluates to* ***"x0"****. The created literal is a perfectly canonical* string *— the failure is faithfulness, not form: the member's runtime value escapes its static type. Witnessed end-to-end (issue report* ***enum-concat-imprecise-fold.md****; SKJS accepts the assignment at the wrong literal type, tsc types* ***E.A*** *as* ***E*** *and rejects). Two repairs fit the current representation: concatenate* ***String(Number(lexeme))*** *— correct JavaScript semantics — or conservatively* reject *a string fold whose numeric operand fails* ***isPreciseJSNumberLiteral****. The numeric fallback cannot be reused: the representation can express an enum member whose numeric value is unknown but has no equivalent for strings. Refusing the constant fold today routes into the computed-member path, which expects a number; adding such a string representation would be a larger design change.*

Excluding the concat arms, the enum machinery is otherwise an upstream *client* of the canonical-literal domain: no enum path creates a lexeme outside the canonical forms already audited, and the faithfulness gap is confined to Finding 21.

impl: `TypeCheckEnum.sk / enumFoldNumericBinop (:131--190), enumNumOrUnknown (:123--126), EFStr concat arms (:439--457), EMVLiteral construction (:220--265)`

7.5 Validation of over-lexed input

The numeric-token loop in the lexer accepts a strict superset of the NumericLiteral grammar — any run over $\{$`0-9`, `a-f`, `A-F`, `e`, `E`, `.`, `x`, `X`, `b`, `B`, `o`, `O`, `_`$\}$ — so lexemes like `1.2.3`, `0x`, or `1__2` reach `getNumber`. `validateJSNumericLiteral`/`validateJSBigIntLiteral` reject these non-grammar lexemes with a diagnostic before `normalizeJSNumber`/`normalizeJSBigInt` runs. Signed exponents (`1e-5`) never arrive as one token (the loop stops at `-`/`+`), so the normalizer's input grammar has unsigned exponents only, while its *output* may contain signed ones (`"1e-7"`), which the lexer does not re-tokenize as a single numeric token. The string scanner analogously accepts any escape-paired body (it advances past backslash pairs without judging them, `Lexer.sk:67-110`, string scans :505–546) and defers judgment to `decodeJSStringLiteral`, which rejects the invalid escapes (§7.2).

impl: `Lexer.sk / numeric scan (:350), string scans; ParserUtils.sk / getNumber, getBigInt, unquoteString`

7.6 User-visible pretty-printing

Literal types render through `renderPrimLiteral`: string values are double-quoted with backslashes, quotes, and control characters re-escaped for display (stored values are decoded, so a value may contain raw control characters that diagnostics must not emit); all other kinds print their stored lexeme bare. Diagnostics and hovers therefore show canonical spellings: `0o0 | 1` prints as `0 | 1`, `0x1 | 1` collapses to `1`, and `"\x61"` prints as `"a"`. This matches tsc's rendering. Two corners: (1) large/small magnitudes print in canonical scientific form (`1e+21`, `1e-7`) — valid TS literal spellings, though SKJS's own lexer cannot re-lex the signed-exponent ones (§7.5); (2) overflowed literals print as `Infinity`, which is *not* a round-trippable literal-type spelling.

impl: `TTypePp.sk / renderPrimLiteral (:729--755), appendControlEscape`

7.7 Interaction with `PrimLiterals` operations

`union/inter/diff/contains/xxhash` are lexeme-level; under the canonical-storage invariant they are exact value-set operations (Corollary 18). In particular `sameConstant` and structural type hashing are value-respecting, and `TFields::isTuple`'s `toIntOption` on the `length` literal resolves radix-spelled lengths through their canonical form (canonical `"2"` succeeds where a raw `"0x2"` would fail `toIntOption`).

impl: `PrimLiterals.sk; TType.sk / TPrim::sameConstant, TFields::isTuple`

7.8 Numeric property names

Numeric *property names* take the same normalization path (`ParserDefs.sk/getNameOrStringOrNumber`, :72) and end up as field-name strings compared against literal-type lexemes (`FieldKey::ofSType`; `TFields::maybeGet`; `namedFieldsForKeyKind` filters names by `isJSNumberKeyName`). TS defines a numeric property name as `String(Number(name))`, i.e. exactly $\mathsf{Canon}_{\mathtt{num}}$'s finite part. For coherence, a declaration `{ 0x10: T }` must store the field name `"16"` so that the canonicalized access key `obj[0x10]` (literal type `16`) resolves to it — otherwise declaration-side and use-side spellings would sit permanently disconnected. Parser-side normalization of the declaration name is what keeps the two sides aligned; quoted property names decode their escapes on the same path (§7.2), keeping `{"\x61": T}` and `obj.a` aligned too.

impl: `ParserDefs.sk / getNameOrStringOrNumber; TType.sk / FieldKey::ofSType (:1584), namedFieldsForKeyKind`

7.9 Falsiness on canonical lexemes

Because stored lexemes are canonical, the falsiness test reads them directly. `isFalsy` (`TType.sk:1548`) compares against the canonical falsy sets of Theorem 15: `TNum() -> literal == "NaN" || literal == "0"` (the `"NaN"` arm is defensive, though Theorem 4 shows no site creates it); `TBigInt() -> literal == "0n"`; `TString() -> literal == ""` (exact, since the stored lexeme is the decoded value); other kinds compare against their own canonical falsy lexemes. There is no separate zero-spelling predicate: a single canonical string per value makes the equality test exact — on the canonical range (Definition 11); the beyond-precision residue is Finding 22.

8 Findings from Failed and Partial Proof Attempts

Per the series contract, unprovable steps are reported as findings, not silently narrowed away.

Finding 22 (Beyond-precision spellings: L-LIT(ii)/(iii) fail for kind `num`, witnessed)**.** *Attempting to extend Theorems 14/15 from canonical forms to* all *stored* `num` *lexemes fails on the beyond-precision residue of Definition 11. Distinct stored lexemes can alias one double —* `9007199254740993` *and* `9007199254740992` *are both* 2^{53}*;* `0x20000000000001` *spells the same value in hex;* `1e309` *and* `2e309` *are both* `Infinity` *— so injectivity (ii) fails, and through* `PrimLiterals::inter`*'s lexeme intersection (Corollary 18's parenthetical) equality narrowing types a reachable branch* `never`*: with the two* 2^{53} *spellings, an accepted* `===`*-true branch is live at runtime and a number flows into* `string` *(executed witness: issue report* `issues/beyond-precision-numeric-literals.md`*). Falsiness (iii) fails on the same residue: the normalizer rewrites a longhand* 10^{-400} *decimal to the lexeme* `"1e-400"`*, whose IEEE value underflows to* $+0$ *— falsy at runtime, yet* `isFalsy("1e-400")` *reads it truthy, so with* `x:` 10^{-400}`-longhand | ""` *the* `else` *of* `if (x)` *narrows to* `""` *and* `const z: "" = x` *is accepted while that branch runs at* `x = 0` *(a number). The comparison* lint *already knows the aliasing —* `checkComparisonOverlap` *is silenced for imprecise operands by*

`hasImpreciseNumberLiteral` (`TypeCheckExpr.sk:1656,1704`), its comment citing the 2^{53} pair — and the template-literal-type reduction refuses imprecise spellings outright (§7.3), but the narrowing and intersection path carries no precision guard, and `isFalsy`'s `num` comment asserts beyond-precision literals "can never be zero" (`TType.sk:1546-1547`), which the underflow rewriting falsifies. Both witnesses are runnable; issue report: `issues/beyond-precision-numeric-literals.md`. The suite pin `tests/expect-error/number-literal-precision.ts` records the never-merge behavior itself; its header's soundness rationale does not survive these witnesses.

impl: `JSNumericString.sk / normalizeJSNumber (:492), normalizeDecimalJS (:563), isPreciseJSNumberLiteral (:87); TypeCheckExpr.sk / hasImpreciseNumberLiteral; TType.sk / isFalsy (:1548--1556)`

Finding 23 (`isJSNumericString` enforces the ECMA-262 magnitude bounds, but does not realize the round-trip)**.** *The finite part of* Canon_{num} *(Definition 8) is the round-trip characterization* `String(Number(s))` $= s$. *`isJSNumericString` enforces its* magnitude bounds *on exponent forms: the scientific production* $[-][1\text{-}9](.d^*)e[\pm]e^+$ *carries ECMA-262's side condition (`JSNumericString.sk:208`) that scientific rendering is canonical only when the printed exponent is* ≥ 21 *or* ≤ -7*, so a form such as `"1e+5"` — whose value renders as* `String(Number("1e+5"))` $=$ *`"100000"` — is rejected rather than admitted as a second spelling of the same value. The predicate does* not *fully realize the round-trip: it admits beyond-precision decimals (`9007199254740993`, whose value renders `"9007199254740992"`) and the rewritten over/underflow forms (`"1e+309"` renders `"Infinity"`; `"1e-400"` renders `"0"`) — the collisions `isPreciseJSNumberLiteral (:87)` documents. Injectivity (Theorem 14) holds on the round-trip language, not on everything the predicate accepts; the residue is Finding 22. A separate `isJSNumberKeyName` predicate (`:126`) serves the index-signature clients, so the `keyof` and number-index paths do not create non-canonical number literals from string keys.* Canon_{num} *in this paper is defined by the rendering, not by the predicate, so the theorems are stated independently of the predicate's exact grammar; adversarial probing of the `keyof` and number-index paths found* no soundness leak *through the key paths themselves.*

impl: `JSNumericString.sk / isJSNumericString (bounds :208), isJSNumberKeyName (:126), isPreciseJSNumberLiteral (:87); TEnv.sk:3315`

Finding 24 (The code-point value model, and lone surrogates)**.** *The string results hold on a* scalar-value *domain. An ECMAScript string is a sequence of UTF-16 code units; the decoder's output is a sequence of Unicode code points (a Skip `Char` cannot hold a surrogate), and the two representations are in bijection exactly on sequences with no lone surrogate — each astral code point corresponds to exactly one surrogate pair, so injectivity (Theorem 14) transports across the encoding. The decoder maintains the restriction by construction: `\uHHHH` high surrogates must be completed by a low surrogate (the pair becomes one code point), and lone-surrogate escapes are rejected (`UnrepresentableSurrogateEscape`), where tsc accepts them as code units. The divergence is sound — a program is rejected, never mistyped — but it is a deliberate strictness: a literal type for a lone-surrogate string cannot be expressed in SKJS. A runtime string containing a lone surrogate (constructible via `String.fromCharCode`) inhabits `string` and equals no literal type's denotation, which is consistent: strict equality on such a string against any stored literal is `false` at runtime, and the checker's lexeme comparison says the same.*

impl: `JSStringEscape.sk / decodeUnicodeEscape (:196--253); tests/expect-error/string-escape-lone-surrogate-prettier-`

Finding 25 (Why normalization is at construction, not at use sites)**.** *The alternative — keep raw lexemes and parse or decode to values inside `isFalsy`, `inter`, `diff`, `sameConstant`, `xxhash`, subtyping, and printing — is inferior. Many operations already compare literals, and the list grows (any consumer of `PrimLiteral.val` reading the raw lexeme would revert to lexeme semantics), the*

hash requires a canonical key anyway (hashing raw spellings would break hash-consistency of semantically equal types), and per-use parsing would re-do work the constructor does once. Normalize-at-construction keeps the invariant local and makes every downstream operation correct by Corollary 18 without per-site logic — the same argument for both the numeric normalizer and the string decoder, which accordingly live at the same parser choke point (§7.1, §7.2).

9 Status and Registry

9.1 L-LIT at the pin

Item	At the pin
L-LIT(i)	holds (δ_k concrete per kind, §4; NaN-freeness of $\mathsf{Lit}_{\texttt{num}}$ a theorem, Thm. 4; the string decoder realizes $\delta_{\texttt{str}}$ on everything it accepts, §7.2)
L-LIT(ii)	holds for str/bool/sym and for within-precision num/bigint (injectivity over canonical forms, Thm. 14; string storage is canonical by construction, §7.2); fails for beyond-precision `num` spellings, witnessed end-to-end (Finding 22)
L-LIT(iii)	holds for str/bool/sym and for within-precision num/bigint (Thm. 15); fails for beyond-precision `num` (underflow lexeme `1e-400` denotes +0, Finding 22)
Residue lemma (main paper)	discharged over canonical forms (Cor. 16); at the pin the caveat persists only for beyond-precision `num`
rmc exactness (main paper)	over canonical forms (Cor. 17); same caveat
Storage	parse-time `normalizeJSNumber`/`normalizeJSBigInt` and `decodeJSStringLiteral` (= canon_k of §5); property names normalize and decode via the shared parser helpers (§7.8); template-type reduction and enum folding create only canonical lexemes (§7.3, §7.4 — canonical in *form*; the concat arms' value-faithfulness gap is Finding 21); the `isJSNumericString` thresholds (Finding 23) are enforced. Exception: beyond-precision `num` literals (round-trip failures: 16–17-digit decimals, radix $> 2^{53}$, over/underflow exponent forms) are stored non-canonically and never merged, so the canonical-storage invariant of Definition 11 is only *partial* for `num` — the unsound consequences are Finding 22

L-LIT(ii)/(iii) hold exactly where the canonical-storage invariant does; the beyond-precision `num` violations are witnessed (Finding 22).

9.2 Registry

The authoritative, current statuses are maintained in the registry (`docs/ledger.yaml`), rendered as the main paper's Assumptions and Proof Obligations tables; this paper is cited there as evidence. The registry rows it supports:

Row	Statement	Status
L-LIT	Literal spellings: (i) δ is a function; (ii) injectivity modulo $\approx$ after normalization; (iii) falsiness-exactness of the lexeme test.	VIOLATED
L-LIT-FOLD	A literal synthesized by the enum constant folder denotes the member's runtime value.	VIOLATED

Imports (review contract). The named statements this paper assumes from elsewhere. A reviewer treats each as an axiom, checking citations against the registry's canonical statement; no other paper's internals are in scope:
None beyond the main paper's definitions.

9.3 Registry alignment

The registry records, consistent with this paper:

1. **L-LIT status.** The row's status note follows this paper's per-kind, per-precision split: Theorems 14/15 over canonical forms; the string kind and the within-precision num/bigint range canonical by construction (Section 7); the beyond-precision `num` residue carrying the witnessed violation (Finding 22, with its issue report). The row anchors `normalizeJSNumber`/`normalizeJSBigInt`, `isFalsy`, and the string-escape decoding (`JSStringEscape.sk`).

2. **Main paper, Definition "Literal Denotation".** The premise "$\mathsf{Lit}_{\texttt{num}}$ contains no NaN spelling" is a citation of Theorem 4.

9.4 Pinned tests and executable witnesses

The automated suite pins the canonicalization behaviors on both sides. Numeric: `acceptOctalZero` (`tests/projects/truthiness-safe/src/conditions.ts`, :109), whose `0o0` union member and else-branch annotation both normalize to `0`; and `tests/expect-error/number-literal-precision.ts`, which records the never-merge treatment of beyond-precision pairs (whose header's soundness rationale Finding 22 refutes). String: a dedicated family under `tests/expect-error/` pins escape decoding and the inputs it rejects — `string-literal-escapes-prettier-ignore.ts`, `string-literal-line-continuation-prettier-ignore.ts`, `string-escape-octal-prettier-ignore.ts`, `string-escape-nonoctal-decimal-prettier-ignore.ts`, `string-escape-bad-hex-prettier-ignore.ts`, `string-escape-bad-unicode-prettier-ignore.ts`, `string-escape-unicode-out-of-range-prettier-ignore.ts`, `string-escape-lone-surrogate-prettier-ignore.ts`, `string-escape-control-char-rendering-prettier-ignore.ts`, `identifier-escapes-prettier-ignore.ts`, and the template variants (`template-literal-escapes-prettier-ignore.ts`, `template-escape-octal-prettier-ignore.ts`).

The corpus's own runnable witnesses (`docs.py repro`) pin the violated side: the beyond-precision spurious-`never` and underflow-falsiness programs (`issues/repros/beyond-precision-spurious-never.ts`, `issues/repros/underflow-zero-truthy.ts`, Finding 22).

Anchor cross-check hooks per the main paper's maintenance appendix: the L-LIT row anchors `normalizeJSNumber`/`normalizeJSBigInt`, `isFalsy`, `JSStringEscape.sk`, and `TypeCheckEnum.sk`'s fold outputs (canonical in form); the L-LIT-FOLD row also anchors `TypeCheckEnum.sk`, for the orthogonal value-faithfulness claim.